

TP n°3 : MapReduce

Matière : Introduction au Big Data

Classe : 1ère année MPSD

Enseignant TP : Mohamed Anouar DAHDEH



Objectif: Découvrir les jobs **MapReduce**

Introduction

MapReduce essaie de répondre aux problématiques amenées par les Big Data, qui sont des données:

- volumineuses,
- variées,
- complexes,
- qui changent rapidement.

MapReduce résout le problème en divisant les tâches de traitement en plus petites parties (64 Mo) et en les assignant à plusieurs ordinateurs.

- A la fin du traitement, les résultats sont collectés à un seul endroit et intégrés pour former le résultat de traitement.

MapReduce est utilisé dans les cas suivants

- Pour des données non structurées et sans schéma (pas de langage trop lourd comme le SQL, XML).
- Pour de très grands clusters.
- Traitement et analyse de type batch.

1- Principe du Map

Dans l'étape **Map**, le but est de partir d'un couple <clé, valeur> et d'y associer de nouveaux couples <clé, valeur>.

```
//En pseudo code cela donnerait
map(string key, string value) {
    // key: document name
    // value: document contents
    for each word w in value
        emit <w, 1>
}
```

Note

Le nombre de tâches **Map** ne dépend pas du nombre de nœuds, mais du nombre de blocs de données en entrée. Chaque bloc se fait assigner une seule tâche Map. De plus, toutes les tâches Map n'ont pas besoin d'être exécutées en même temps en parallèle.

2- Principe du Reduce

Dans l'étape **Reduce** le but est d'associer toutes les valeurs correspondantes à la même clé.

On souhaite donc rassembler tous les couples <clé, valeur>.

```
//En pseudo code cela donnerait
reduce(string key, list value) {
    // key: word
    // value: list of each word appearance
    sum = 0
    for each v in value
        sum = sum + v
    emit <key, sum>
}
```

Note

Pour le traitement, les tâches **Reduce** suivent le même schéma que les tâches **Map**. Elles n'ont pas à s'exécuter parallèlement et dès qu'un nœud fini son traitement un autre lui est aussitôt assigné.

3- Implémentation dans Hadoop

- Écrit en java à l'origine par Yahoo repris par Apache.
- La gestion des fichiers se fait par **Hadoop : HDFS**
- Les jobs déjà incluent dans **Hadoop** :
 - ❖ **aggregatewordcount** : Compte les mots des fichiers en entrée.

- ❖ **aggregatewordhist** : Traite l'histogramme des mots des fichiers en entrée.
- ❖ **bbp** : Utilise la méthode Bailey-Borwein-Plouffe pour calculer les chiffres exacts de PI.
- ❖ **dbcount** : Compte le nombre de vue sur une page de BDD.
- ❖ **distbbp** : Utilise la formule BBP-type pour calculer les chiffres exacts de PI.
- ❖ **grep** : Compte les correspondances avec la reg-ex donné en entrée.
- ❖ **join** : Effectue une jointure sur des ensembles de données triés, également répartis.
- ❖ **multifilewc** : Compte les mots de plusieurs fichiers
- ❖ **pentomino** : Un programme de pose de tuiles pour trouver des solutions aux problèmes pentomino.
- ❖ **pi** : Estime PI en utilisant la méthode Monte-Carlo.
- ❖ **randomtextwriter** : Écrit 10 Go de données textes aléatoires par nœud.
- ❖ **randomwriter** : Écrit 10 Go de données aléatoires par nœud.
- ❖ **secondarysort** : Définit un nouveau tri pour le Reduce.
- ❖ **sort** : Un programme qui trie les données écrites par randomwriter.
- ❖ **sudoku** : Résout des sudoku.
- ❖ **teragen** : Génère des données pour le térasort.
- ❖ **terasort** : Fait le térasort.
- ❖ **teravalidate** : Vérifie les résultats du térasort.
- ❖ **wordcount** : Compte les mots des fichiers en entrée.
- ❖ **wordmean** : Compte la longueur moyenne des mots des fichiers en entrée.
- ❖ **wordmedian** : Calcule la longueur médiane des mots des fichiers en entrée.
- ❖ **wordstandarddeviation** : Calcule l'écart type de la longueur des mots des fichiers en entrée.

4- Découverte des jobs :

➤ Comment utiliser les jobs déjà présents dans **Hadoop** ?

Une fois Hadoop est démarré, il suffit de lancer les exemples déjà présents sous le fichier `hadoop-mapreduce-examples-{version}.jar` du répertoire `/usr/lib/hadoop-mapreduce` à l'aide de la commande :

```
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-mapreduce-examples-{version}.jar ARGUMENT1 ARGUMENT2 ARGUMENT3
```

Note

Dans le cas de notre Cloudera VM le fichier JAR : `hadoop-mapreduce-examples-{version}.jar` correspond au `hadoop-mapreduce-examples-2.6.0-cdh5.13.0.jar`

5- Travail à faire :

Premier Job : Calcul de PI

Ce premier job **MapReduce** permet d'estimer la valeur du nombre pi et ne requiert pas de fichiers. Le programme utilise une méthode statistique Monte-Carlo pour faire l'estimation.

Lancer `hadoop-mapreduce-examples-{version}` avec comme premier argument `pi`, deuxième argument le nombre de map souhaitées (2) et troisième argument le nombre d'échantillons souhaités par map (5).

Deuxième Job : WordCount

A l'aide de **gedit** créer le fichier `NbMots.txt` contenant la phrase suivante :

Tout ce qui est rare est cher, un cheval bon marché est rare, donc un cheval bon marché est cher.

Enregistrer ce fichier sous `/home/cloudera`

Ajouter le fichier `NbMots.txt` dans le système de fichier de **hadoop** sous `/user/cloudera/WordCountJob/input`.

Lancer `hadoop-mapreduce-examples-{version}` avec comme premier argument `wordcount`, deuxième argument le nom du fichier en input (`/user/cloudera/WordCountJob/input`) et le troisième argument : la sortie (`/user/cloudera/WordCountJob/out`).

Visualiser le résultat avec Hue

Troisième Job : WordMean

Lancer `hadoop-mapreduce-examples-{version}` avec comme argument `wordmean` le nom du fichier en input (`/user/cloudera/WordMeanJob/input`) et la sortie (`/user/cloudera/WordMeanJob/out`).

Visualiser le résultat avec Hue

Quatrième Job : Sudoku

				2	8		7	
			3					8
		8			1			4
	4					7		6
	8		7	5	6		4	
5		7					1	
9			8			6		
8					9			
	2		5	4				

Télécharger le fichier `sudoku.dta` dans le système de fichier **hadoop** :

```
/user/cloudera/sudokuJob/input/sudoku.dta
```

Lancer `hadoop-mapreduce-examples-{version}` avec comme premier argument `sudoku` et le deuxième argument : le nom du fichier en input (`sudoku.dta`).